



Software Quality Engineering

Software Testing in Software Quality Assurance

Ms. Areej Bashir

What We'll Cover



Software Testing

Definitions, purpose, and who performs it



SQA Activities for Testing

Verification, validation, and qualification



The Test Plan

Structure, considerations, and IEEE 829 documents



White Box & Black Box Testing

Structural vs. functional test strategies



Implementation of Testing

Running tests, correcting errors, retesting



Alpha & Beta Site Tests

Real-world validation before release

Several Definitions of Testing

“

“Testing is the process of establishing confidence that a program or system does what it is supposed to.”

— Hetzel

“

“Testing is the process of executing a program or system with the intent of finding errors.”

— Myers

“

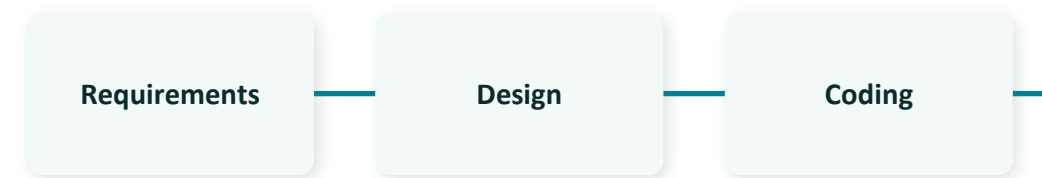
“Testing is any activity aimed at evaluating an attribute or capability of a program or system and determining that it meets its required results.”

— Hetzel, 1983

Testing in the Development Lifecycle

Software testing is one of the most important phases of software development — a process built on well-defined quality control standards, testing methods, strategy, test criteria, and tools.

Engineers perform all types of software testing activities to carry out a complete test process. Testing is the last quality-checking point for software on its production line.



Testing: the final quality checkpoint before release

Who Does Software Testing?



Test Manager

Manages and controls a software test project; supervises engineers in defining a test plan.



Test Engineers & Testers

Define test cases, write test specifications, and run tests.



Independent Test Group

Provides objective, unbiased evaluation independent of the dev team.



Development Engineers

Perform unit tests and integration tests only.



Quality Assurance Group

Performs system testing; defines testing standards and quality control.

Verification, Validation & Qualification

Three process approaches for the evaluation of software products



Verification

Checks whether the examined item correctly and fully implemented the conditions and requirements presented at the beginning of the development phase.



Validation

Checks whether the examined product correctly and fully implemented the relevant specified requirements.

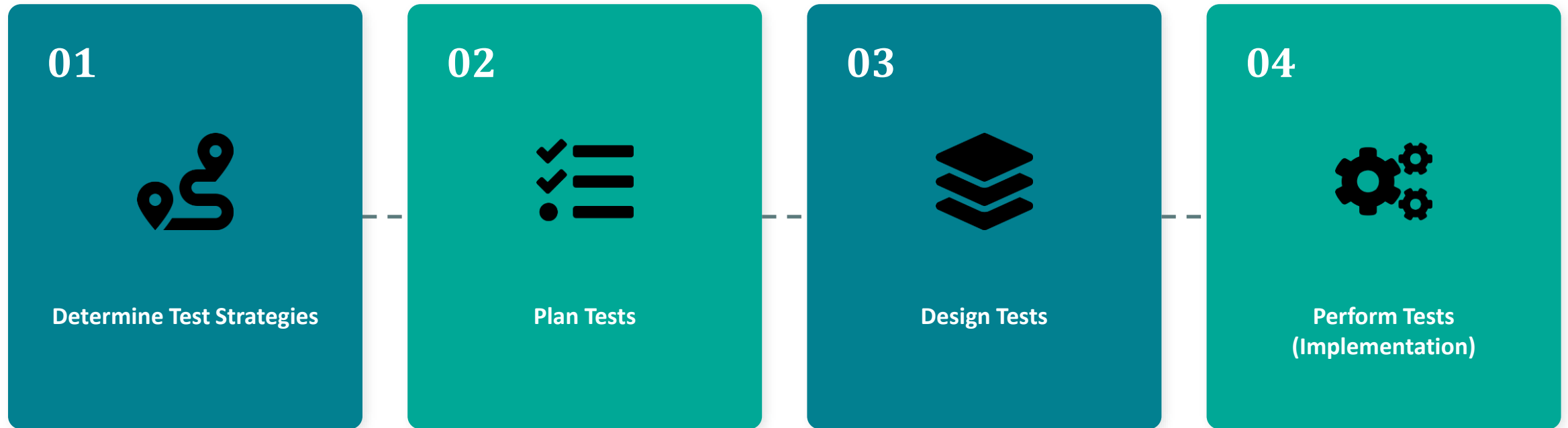


Qualification

Checks whether the examined product correctly and fully implemented professional coding standards, style, structure, and procedures.

Software Testing Resource Utilization

The software testing process moves through four sequential stages:



Defining the Test Plan

Who

What

When

Where

How

Test Plan Considerations

- What are the critical or most complex modules? Make sure they get integration-tested first — they likely deserve white-box attention.
- Where have you had problems in the past?
- Are there third-party delivered components to account for?
- What training is required — formal reviews, testing tools, defect report logging?

8 Types of Testing Documents (IEEE 829)

1

Test Plan

Overall scope, approach, resources, and schedule

2

Test Design Specification

Expected results, pass criteria

3

Test Case Specification

Test data for use in running the tests

4

Test Procedure Specification

How to run each test

5

Test Item Transmittal Report

Reports when components progress between test stages

6

Test Log

Chronological record of test execution

7

Test Incident Report

For any failed test: actual vs. expected result

8

Test Summary Report

Management-level report

Test Plan Contents — IEEE 829 Format

1. Test Plan Identifier

2. References

3. Introduction

4. Test Items

5. Software Risk Issues

6. Features to be Tested

7. Features not to be Tested

8. Approach

9. Item Pass / Fail Criteria

10. Suspension Criteria & Resumption Reqs.

11. Test Deliverables

12. Remaining Test Tasks

13. Environmental Needs

14. Staffing and Training Needs

15. Responsibilities

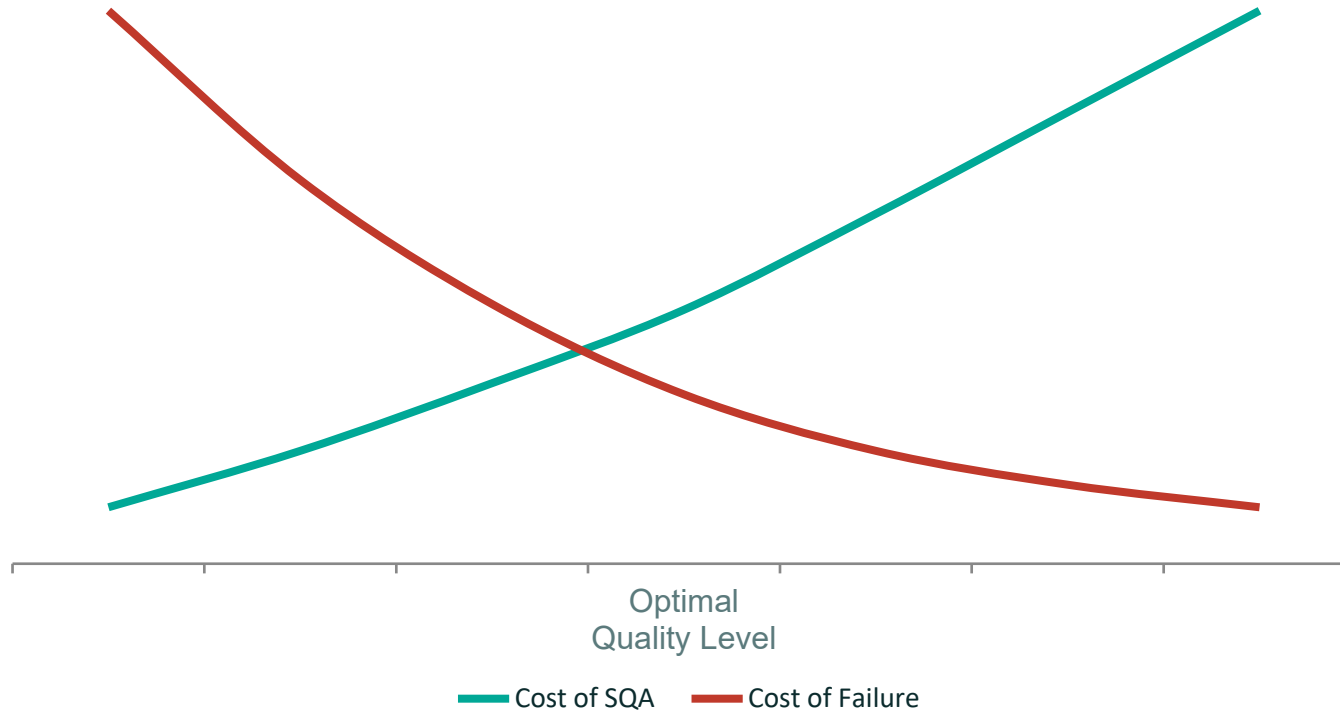
16. Schedule

17. Planning Risks and Contingencies

18. Approvals

19. Glossary

When Is More Testing Not Cost-Effective?



Reading the Curve

- As SQA activity increases, the cost of SQA rises — but the cost of failure falls.
- Total cost (SQA + Failure) is lowest at the optimal quality level.
- Testing beyond this point adds cost without a matching drop in failure risk.

The Testing Concept



Black Box (Functionality) Testing

Identifies bugs only according to software malfunctioning as revealed in erroneous outputs. When outputs are correct, black box testing disregards the internal path of calculations and processing performed.



White Box (Structural) Testing

Examines internal calculation paths to identify bugs. Also called “glass box testing” — a name that better expresses its basic characteristic: investigating the correctness of code structure.

Test Classification According to Requirements

Test Classification	White Box	Black Box
Correctness tests	✓	✓
User manuals tests		✓
Availability (reaction time) tests		✓
Reliability tests		✓
Stress tests (load & durability)		✓
Software system security tests		✓
Usability tests (training & operational)		✓
Maintainability correctness tests	✓	✓
Flexibility / Testability / Portability tests		✓
Reused software correctness tests	✓	✓
Interoperability tests (software & equipment)		✓

Putting the Concepts into Practice



White Box Testing Realization

Requires verification of every program statement and comment. To perform data-processing and calculation correctness tests, every computational operation in each test-case path must be examined — verifying correctness for that path, but not necessarily others.

Path Coverage

Measured by the percentage of possible paths covered by tests.

Line Coverage

Measured by the percentage of program code lines covered by tests.



Black Box Testing Realization

Allows us to perform correctness tests and most other classes of tests, based only on processing outputs. Success relies solely on selecting appropriate test cases that uncover errors across the wide variety of program paths.

While the effectiveness of black box testing relies on a wider variety of test cases, its efficiency depends on decreasing the number of test cases — a lengthy search for the optimal, economic size of test-case sets.

Requirement-Driven Software Testing

Operation factor category — connecting quality requirements to test classes

Quality Requirement Factor	Quality Requirement Sub-factor	Test Classification
Correctness	Accuracy & completeness of outputs / data	Correctness tests
	Accuracy & completeness of documentation	User manuals tests
	Availability (reaction time)	Availability tests
Reliability	—	Reliability tests
Efficiency	—	Stress tests (load & durability)
Integrity	—	Software system security tests
Usability	Training usability	Training usability tests
	Operational usability	Operational usability tests

The Implementation Process

Testing implementation typically consists of a repeating cycle:



1

Tests

Carried out by running test cases per the test procedures. Documentation of the procedures and test-case data form the “Software Test Description” (STD).



2

Correction of Errors

Carried out by developers under a highly controlled process, with follow-up to ensure every error listed in the STR is corrected.



3

Retesting (Regression)

Verifies that previously detected errors were properly corrected and that no new errors entered the system. This cycle often repeats 2–4 times.

Automated vs. Manual Testing, by Phase

Testing Phase	Automated Testing		Manual Testing	
Test planning	Manual	Preparing the test plan	Manual	Preparing the test plan
Test design	Manual	Preparing the test database	Manual	Preparing the testing procedure
Preparing test cases	Manual	Preparing & recording test cases	Manual	Preparing test cases
Performance of tests	Automated	Computerized running of tests	Manual	Performing tests with testers
Regression tests	Automated	Computerized running of tests	Manual	Performing tests by testers
Test log & reports	Automated	Computerized output	Manual	Prepared by the testers

Validating with Real Users



Alpha Site Tests

Performed by potential users at the developer's site on a new software package. Customers apply the software to their organization's specific requirements, examining it from angles the testing team may not have expected — revealing errors only real users can find.



Beta Site Tests

Much more common than alpha tests. Once an advanced version is available, the developer offers it free-of-charge to potential users, who install it at their own “beta site” and report all errors found during trials or regular use — sometimes involving hundreds or thousands of participants.



Review of Code Listing

Code inspections and walkthroughs are performed on unit and integration software prior to unit/integration tests — successfully identifying a significant share of software errors. Also applied to maintenance-phase code.



Software Qualification Reviews

Check whether code or documents fulfill structure standards, coding style procedures, and documentation standards — including programmer manuals and help sections.

Anatomy of a Test Case

A test case is simply a pair: <input, expected outcome>

Test Case ID

Unique identifier for the test case

Test Data / Input

Data, or links to data, used while conducting the test

Actual Result

Filled in after the test is executed

Description / Summary

Why the test is being performed

Expected Result

Detailed expected outcome, including messages or errors

Status / Verdict

Pass if actual matches expected; otherwise Fail

Test Case Example — Calculator

ID	Description	Input	Expected Result
TC001	Addition of two positive numbers	$5 + 3$	Result = 8
TC002	Subtraction with a negative result	$4 - 9$	Result = -5
TC003	Multiplication with zero	0×7	Result = 0
TC004	Division of two numbers	$20 \div 5$	Result = 4
TC005	Division by zero	$9 \div 0$	Display error: Division by zero

Precondition for all cases above: Calculator is ON

KEY TAKEAWAYS

Testing Turns Confidence into Evidence

- Testing is the final quality checkpoint — verification, validation, and qualification all play a role.
- A solid test plan (who, what, when, where, how) drives consistent, repeatable testing.
- White box and black box strategies are complementary, not competing, approaches.
- Alpha, beta, and regression cycles bring real-world confidence before release.